

# AutoChip Report

## 1. Setup and Method

I used the provided ChipChat/AutoChip notebook in Google Colab. The LLM model was gpt-4o-mini and the API key was loaded from Colab Secrets. I used Icarus Verilog (iverilog and vvp) to compile and simulate each generated RTL design.

For each example, I first used the natural-language specification in the notebook, then generated or adjusted the Verilog code, and finally checked it with the provided testbench. When a design did not match the testbench, I used the error or simulation result to revise the RTL.

## 2. Results Summary

| Example            | Type                        | Final status      |
|--------------------|-----------------------------|-------------------|
| binary_to_bcd      | Combinational converter     | Passed            |
| sequence_detector  | FSM sequence matching       | Passed            |
| shift_register     | Sequential shift register   | Passed            |
| abro_state_machine | ABRO FSM                    | Passed            |
| dice_roller        | Pseudo-random behavior      | Passed            |
| lfsr               | 8-bit LFSR                  | Passed            |
| sequence_generator | Pattern generator           | Passed            |
| traffic_light      | Traffic light FSM           | Passed            |
| 8-bit CPU          | Accumulator-based processor | Passed smoke test |

## 3. Eight Tutorial Examples

### Example 1 - binary\_to\_bcd

**Specification:** Convert a 5-bit binary input to an 8-bit BCD output.

**Trajectory:** The initial generated version had interface mismatches with the testbench. I corrected the module/port names and used the test feedback to reach the final version.

**Verification:** ``cd binary_to_bcd/ && iverilog -g2012 -o binary_to_bcd.vvp binary_to_bcd.v binary_to_bcd_tb.v && vvp binary_to_bcd.vvp``

**Result:** All test cases passed.

**Main change:** The important change was matching the exact testbench interface and keeping the BCD conversion behavior correct.

### Example 2 - sequence\_detector

**Specification:** Detect the ordered 3-bit sequence 001, 101, 110, 000, 110, 110, 011, 101.

**Trajectory:** The first generated designs had syntax/interface issues and then a timing/logic issue at the last value. The final FSM used the exact testbench ports and timing.

**Verification:** ``cd sequence_detector/ && iverilog -g2012 -o sequence_detector.vvp sequence_detector.v sequence_detector_tb.v && vvp sequence_detector.vvp``

**Result:** All test cases passed.

**Main change:** Tool feedback was most useful here because it changed both the interface and the FSM output timing.

### Example 3 - shift\_register

**Specification:** Create an 8-bit shift register with clock, active-low reset, serial data input, and shift enable.

**Trajectory:** The normal first implementation did not match the provided testbench timing/expected sequence. I adjusted the implementation to match the verification behavior.

**Verification:** ``cd shift_register/ && iverilog -o shift_register.vvp shift_register.v shift_register_tb.v && vvp shift_register.vvp``

**Result:** All test cases passed.

**Main change:** The final version was mainly changed to match the testbench timing and expected output sequence.

#### Example 4 - abro\_state\_machine

**Specification:** Implement an ABRO-style FSM with one-hot state output.

**Trajectory:** The LLM produced an FSM structure, and I adjusted the state update timing and interface to match the supplied testbench.

**Verification:** ``cd abro_state_machine/ && iverilog -o abro_state_machine.vvp abro_state_machine.v abro_state_machine_tb.v && vvp abro_state_machine.vvp``

**Result:** All test cases passed.

**Main change:** The state encoding and update timing were the main parts that needed attention.

#### Example 5 - dice\_roller

**Specification:** Simulate 4-, 6-, 8-, and 20-sided dice using a 2-bit die select and a roll input.

**Trajectory:** The generated design used pseudo-random behavior. The final RTL used an 8-bit LFSR and mapped its value to the selected die range.

**Verification:** ``cd dice_roller/ && iverilog -o dice_roller.vvp dice_roller.v dice_roller_tb.v && vvp dice_roller.vvp``

**Result:** Testbench completed successfully for all four die selections.

**Main change:** The important change was keeping every generated number inside the correct die range.

#### Example 6 - lfsr

**Specification:** Create an 8-bit LFSR with initial state 10001010 and the requested taps.

**Trajectory:** The final version used the expected XOR taps and updated at the clock edge that matched the reference testbench timing.

**Verification:** ``cd lfsr/ && iverilog -o lfsr.vvp lfsr.v lfsr_tb.v && vvp lfsr.vvp``

**Result:** Simulation successful. All test cases passed.

**Main change:** The feedback/timing relation to the testbench was the main adjustment.

#### Example 7 - sequence\_generator

**Specification:** Generate AF, BC, E2, 78, FF, E2, 0B, 8D repeatedly while enabled.

**Trajectory:** The final RTL used a 3-bit index and a case statement to generate the eight values. The index update was adjusted so the first checked value stayed AF.

**Verification:** ``cd sequence_generator/ && iverilog -o sequence_generator.vvp sequence_generator.v sequence_generator_tb.v && vvp sequence_generator.vvp``

**Result:** Sequence generation, repetition, and disable tests all passed.

**Main change:** The most important fix was the index timing so the output did not advance one position too early.

#### Example 8 - traffic\_light

**Specification:** Traffic light FSM: red for 32 cycles, green for 20 cycles, yellow for 7 cycles, then repeat.

**Trajectory:** The LLM generated an FSM idea and the final design used a state register plus a counter for each light duration.

**Verification:** ``cd traffic_light/ && iverilog -o traffic_light.vvp traffic_light.v traffic_light_tb.v && vvp traffic_light.vvp``

**Result:** All four transition tests passed.

**Main change:** The counter limits were matched to the exact cycle counts required by the testbench.

## 4. 8-bit CPU Example

For the CPU part, I started from the small accumulator-based 8-bit processor idea in the assignment. Instead of asking for Verilog immediately, I first asked the LLM to propose a simple architecture and ISA. Then I gave human feedback to reduce it to a one-byte instruction format with a small accumulator datapath.

**Final simplified ISA:** NOP, LDI, ADD, SUB, AND, OR, XOR, LSL, LSR, JNZ, OUT, and HALT.

The generated module used an external 8-bit instruction input, an 8-bit program counter, an 8-bit accumulator, an output port, and a halt flag. This kept the design small and avoided an internal instruction memory.

**Verification:** ``iverilog -g2012 -o cpu_8bit/cpu_8bit.vvp cpu_8bit/cpu_8bit.v cpu_8bit/cpu_8bit_tb.v`` followed by ``vvp cpu_8bit/cpu_8bit.vvp``.

**Result:** PASS: CPU test completed successfully. Final accumulator = 7 and out\_port = 7.

The CPU test was only a small smoke test, so it does not prove every instruction or branch case. The harder part compared with the small examples was deciding the ISA and control behavior while keeping the design simple. A more complete version would need more tests for JNZ, reset behavior, edge cases, and longer instruction sequences.

## 5. Conclusion

All eight tutorial examples passed their provided simulations, and the 8-bit CPU also passed the basic test. The main lesson was that LLM-generated Verilog often needed small fixes for exact module interfaces, state timing, and testbench expectations. Compiler and simulation feedback were useful for finding these problems quickly.